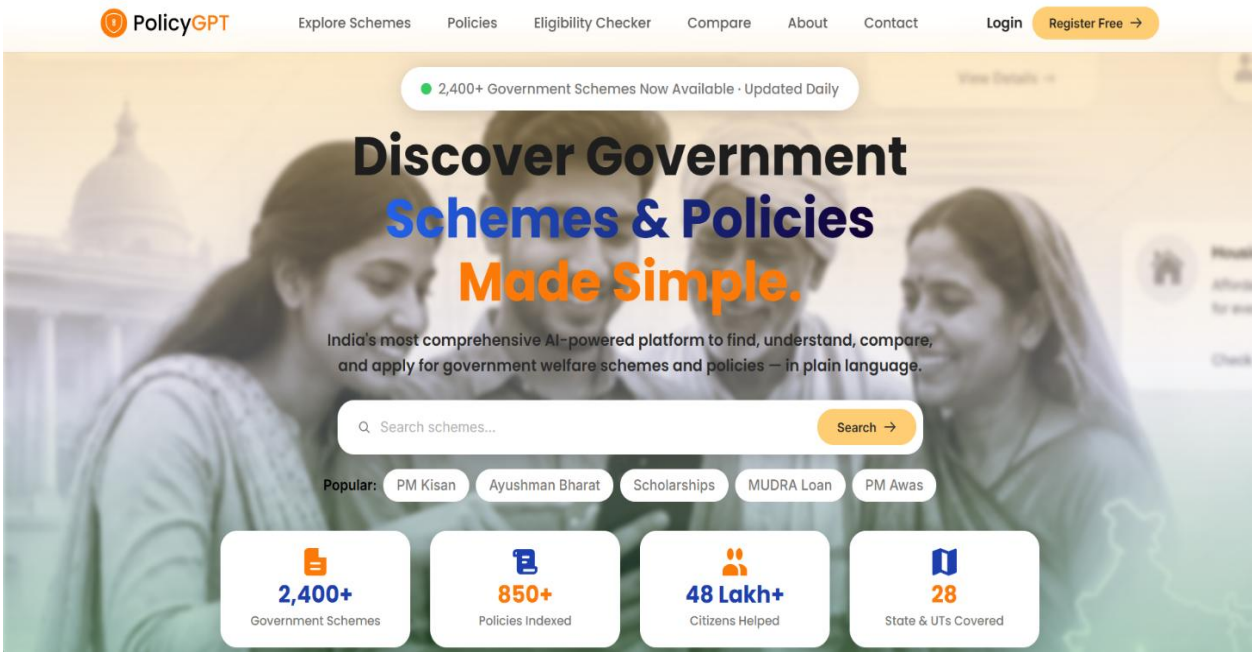
# POLICYGPT

*Government Policy & Public Scheme Intelligence Platform*

Making government policies and public welfare schemes easier to discover, understand, and access.




**Project Documentation**

Infosys Springboard Internship 7.0

Status: Active Development

# Table of Contents


Table of Contents ..... 2
1. Project Overview ..... 4
2. Problem Statement ..... 4
    2.1 Difficulty Discovering Relevant Schemes ..... 4
    2.2 Complex and Lengthy Information ..... 4
    2.3 Difficulty Checking Eligibility ..... 4
    2.4 Lack of Easy Comparison ..... 4
    2.5 Lack of Centralized Management ..... 4
    2.6 Need for Content Review ..... 4
3. Proposed Solution ..... 5
4. Project Objectives ..... 5
5. System Overview & User Roles ..... 5
    5.1 Citizen ..... 5
    5.2 Administrator ..... 6
    5.3 Volunteer / Reviewer ..... 6
6. Roles and Permissions ..... 6
7. Key Features ..... 7
    7.1 Authentication & Role-Based Access Control ..... 7
    7.2 Policy & Scheme Search ..... 7
    7.3 Scheme Management ..... 7
    7.4 Eligibility Checker ..... 7
    7.5 Scheme Comparison ..... 7
    7.6 Notifications ..... 7
    7.7 Analytics Dashboard ..... 7
    7.8 Reports ..... 7
    7.9 Feedback & Support ..... 7
    7.10 Review Workflow ..... 7
8. Technology Stack ..... 8
    8.1 Frontend ..... 8
    8.2 Backend ..... 8
    8.3 AI & Data Processing ..... 8
    8.4 Development Tools ..... 9
9. System Architecture ..... 9
10. Project Folder Structure ..... 10

**11. Installation and Setup** ..... 11
**11.1 Prerequisites** ..... 11
**11.2 Clone the Repository** ..... 11
**11.3 Backend Setup** ..... 11
**11.4 Frontend Setup** ..... 11
**12. Development Workflow** ..... 11
**Example Commit Messages** ..... 11
**13. Application Workflows** ..... 12
**13.1 Citizen Workflow** ..... 12
**13.2 Volunteer Review Workflow** ..... 13
**14. Application Screens** ..... 14
**14.1 Landing Page** ..... 14
**14.2 Citizen Dashboard** ..... 14
**14.3 Scheme Details** ..... 15
**14.4 Eligibility Checker** ..... 15
**14.5 Scheme Comparison** ..... 16
**14.6 Admin Dashboard** ..... 17
**14.7 Volunteer Dashboard** ..... 18
**15. API Documentation** ..... 18
**16. Error Handling and Troubleshooting** ..... 18
**16.1 Frontend Issues** ..... 18
**16.2 Backend Issues** ..... 19
**16.3 Database Connection Issues** ..... 19
**16.4 Authentication Issues** ..... 19
**17. Future Enhancements** ..... 19
**18. Maintenance and Support** ..... 19
**19. Project Status** ..... 20
**20. Core Contributors** ..... 20
**21. Conclusion** ..... 20
**23. Next Steps for a Fully Detailed Version** ..... 21

# 1. Project Overview

PolicyGPT is a full-stack web platform built to simplify access to government policies, welfare schemes, notifications, and other public information. It centralizes information that is otherwise scattered across multiple sources and lengthy documents, making it easier for citizens to discover and understand the schemes relevant to them.

The platform provides dedicated workspaces for three user roles — Citizens, Administrators, and Volunteers/Reviewers — supported by role-based access control and a structured content-management workflow.

# 2. Problem Statement

Government policies, welfare schemes, notifications, and public information are often scattered across different sources and presented in lengthy or complex formats. This makes it difficult for citizens to efficiently discover and understand information relevant to their needs. PolicyGPT addresses the following core problems:

### 2.1 Difficulty Discovering Relevant Schemes

Citizens often need to visit multiple government platforms or sift through large volumes of information to find schemes relevant to them.

### 2.2 Complex and Lengthy Information

Scheme and policy documents are frequently detailed and difficult for the average user to quickly understand.

### 2.3 Difficulty Checking Eligibility

Users often struggle to determine whether they meet the eligibility requirements of a given scheme.

### 2.4 Lack of Easy Comparison

When several schemes offer similar benefits, citizens find it hard to compare eligibility, benefits, and departments side by side.

### 2.5 Lack of Centralized Management

Managing schemes, policies, users, feedback, notifications, and analytics requires a structured administrative system.

### 2.6 Need for Content Review

Government-related information requires an organized workflow so submitted schemes and policies are verified before publication.

## 3. Proposed Solution

PolicyGPT provides a centralized, web-based platform for accessing and managing government policies and public welfare schemes, guiding citizens through a simple end-to-end journey:

**Explore → Search → Filter → Check Eligibility → Compare → Save → Receive Updates → Submit →Feedback**

Administrators manage users, schemes, policies, notifications, analytics, and reports.
Volunteers/Reviewers review submitted schemes and policies, approve content, or suggest changes as part of the content-verification workflow.

The platform combines a MERN-based full-stack architecture with AI-assisted scheme data enrichment using Google Gemini and Google Search grounding.

## 4. Project Objectives

- Centralize government policy and welfare scheme information on a single digital platform.
- Improve accessibility with an easy-to-use, responsive interface.
- Simplify discovery through search and category-based filtering.
- Support eligibility evaluation against citizen-provided information.
- Enable side-by-side comparison of multiple schemes.
- Improve information management for administrators (users, schemes, policies, notifications, analytics, reports, feedback).
- Establish a structured review workflow for volunteers/reviewers.
- Use AI-assisted processing for scheme data enrichment and intelligent handling.

## 5. System Overview & User Roles

### 5.1 Citizen

The Citizen is the primary user of the platform and can:

- Explore government schemes and policies
- Search and filter information
- View detailed scheme information
- Check eligibility

- Compare schemes
- Save relevant schemes
- Receive notifications
- Submit feedback or report issues

### 5.2 Administrator

The Administrator is responsible for managing and monitoring the platform, and can:

- Manage users
- Manage schemes and policies
- Monitor platform activity and review analytics
- Generate reports
- Manage notifications and feedback
- Control overall platform operations

### 5.3 Volunteer / Reviewer

The Volunteer/Reviewer is responsible for reviewing scheme and policy content, and can:

- Review submitted schemes and policies
- Approve content
- Suggest changes
- Participate in the content-verification workflow

## 6. Roles and Permissions

| Role | Main Permissions |
|---|---|
| Citizen | Explore schemes, search information, check eligibility, compare schemes, save schemes, submit feedback |
| Volunteer | Review policies and schemes, approve content, suggest changes, verify content |
| Administrator | Manage users, schemes, policies, analytics, reports, notifications, and overall platform activity |

The platform implements Authentication and Role-Based Access Control (RBAC) to provide role-specific access to different functionalities.

# 7. Key Features

### 7.1 Authentication & Role-Based Access Control

Secure authentication and role-based access for Citizens, Administrators, and Volunteers/Reviewers. Backed by JWT for authentication/authorization and Bcrypt.js for password hashing.

### 7.2 Policy & Scheme Search

Users can search and filter government policies and welfare schemes by category, helping citizens discover information efficiently instead of navigating multiple sources.

### 7.3 Scheme Management

Detailed scheme information including benefits, eligibility, departments, and application guidance, manageable by administrators.

### 7.4 Eligibility Checker

Evaluates citizen-provided information against scheme eligibility requirements to help users determine whether they may qualify.

### 7.5 Scheme Comparison

Allows users to compare multiple schemes side by side to make more informed decisions.

### 7.6 Notifications

Keeps users informed about relevant platform updates and activities via Nodemailer-based email notifications.

### 7.7 Analytics Dashboard

Visual analytics for monitoring platform usage and department-wise information, built with Recharts.

### 7.8 Reports

Administrative users can generate reports to support monitoring, decision-making, and analysis.

### 7.9 Feedback & Support

Citizens can submit feedback and report issues, providing a channel for improving the platform experience.

### 7.10 Review Workflow

A structured workflow allowing reviewers/volunteers to receive content, examine it, approve or suggest changes, and route it back through the administrative workflow.

# 8. Technology Stack

## 8.1 Frontend

| Technology | Purpose |
|---|---|
| React.js | Component-based frontend development |
| Vite | Frontend build tool and development server |
| Tailwind CSS | Responsive UI styling |
| React Router | Client-side routing and protected routes |
| Axios | API communication |
| Recharts | Dashboard analytics and data visualization |

## 8.2 Backend

| Technology | Purpose |
|---|---|
| Node.js | JavaScript runtime |
| Express.js | REST API and server framework |
| MongoDB | Application database |
| Mongoose | Database modeling and interaction |
| JWT | Authentication and authorization |
| Bcrypt.js | Password hashing |
| Nodemailer | Email-based notifications |

## 8.3 AI & Data Processing

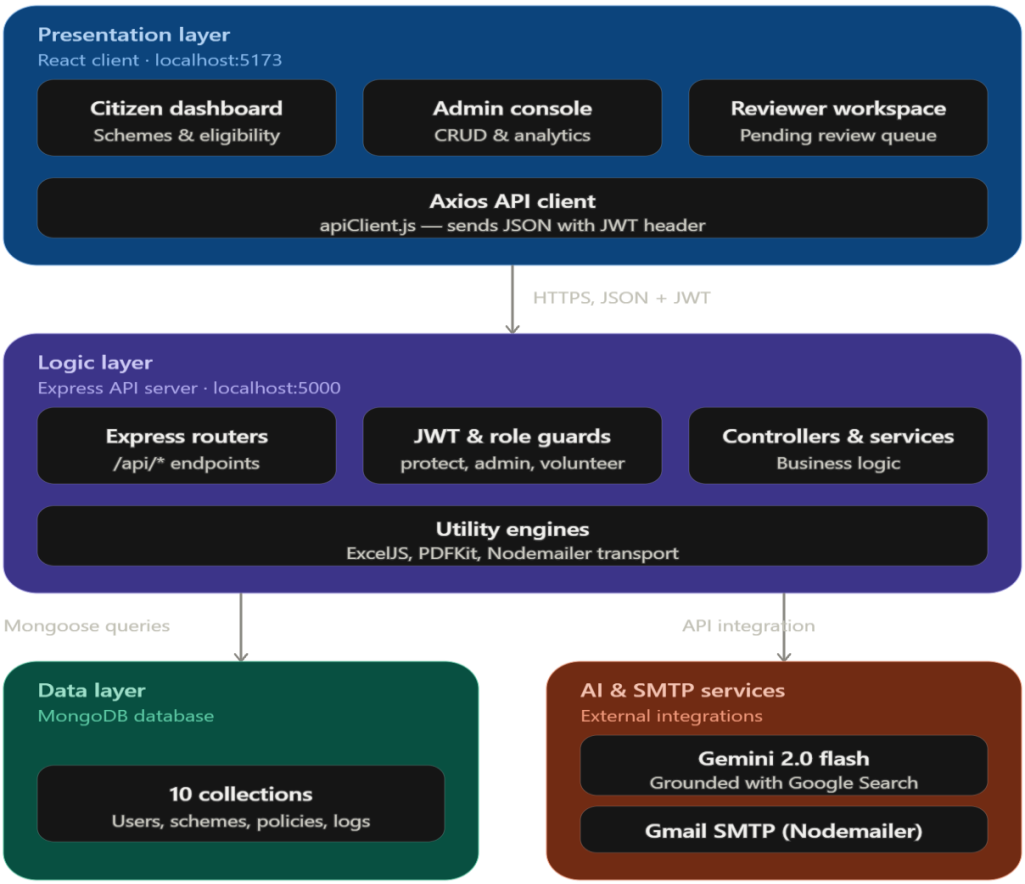
| Technology | Purpose |
|---|---|
| Google Gemini | AI-powered scheme data enrichment and intelligent processing |
| Google Search Grounding | Search grounding used with Gemini for scheme data enrichment |

### 8.4 Development Tools

The project uses Git, GitHub, Postman, Figma, and Visual Studio Code to support version control, collaboration, API testing, UI/UX design, and development.

## 9. System Architecture

High-level architecture of PolicyGPT, showing how Users, the Frontend Client, the Backend Server, the Database, and AI Processing interact:




**System Architecture Diagram**

*Reference architecture (for redrawing as a diagram):*

```
Users (Citizen / Administrator / Volunteer)
        │
        ▼
Frontend Client — React + Vite, Tailwind CSS, React Router, Axios, Recharts
        │  HTTP / API requests
        ▼
Backend Server — Node.js + Express.js
  Controllers · Middleware · Services · Authentication & RBAC
        │                              │
        ▼                              ▼
MongoDB (Users, Schemes,        AI Processing
Policies, Notifications,        (Google Gemini +
Feedback)                       Search Grounding)
```

## 10. Project Folder Structure

```
PolicyGPT/
│
├── client/                # React + Vite frontend
│   ├── public/
│   └── src/
│       ├── assets/        # Images, icons, static assets
│       ├── components/    # Reusable UI components
│       ├── context/       # Global application state
│       ├── hooks/         # Custom React hooks
│       ├── layouts/       # Application layouts
│       ├── pages/         # Route-level pages
│       ├── routes/        # Routing configuration
│       ├── services/      # API service layer
│       ├── utils/         # Utility functions
│       ├── App.jsx
│       └── main.jsx
│
├── Server/                # Node.js + Express backend
│   ├── config/            # Database and environment configuration
│   ├── controllers/       # Request handling and business logic
│   ├── middleware/        # Authentication and authorization
│   ├── models/            # MongoDB / Mongoose models
│   ├── routes/            # API route definitions
│   ├── scripts/           # Seed and utility scripts
│   ├── services/          # Application services
│   ├── utils/             # Helper utilities
│   └── server.js
│
├── docs/                  # Detailed project documentation
├── README.md
└── package.json
```

# 11. Installation and Setup

### 11.1 Prerequisites

- Node.js 18 or above
- npm
- MongoDB (local instance or MongoDB Atlas)
- Git

### 11.2 Clone the Repository

```
git clone <repository-url>
cd PolicyGPT
```

### 11.3 Backend Setup

```
cd Server
npm install
npm run dev
```

### 11.4 Frontend Setup

```
cd client
npm install
npm run dev
```

The application will be available at the local URL provided by Vite (typically http://localhost:5173).

# 12. Development Workflow

PolicyGPT follows a collaborative Git-based development approach using branches, commits, pull requests, code reviews, and merging:

```
Feature / Bug → Create Branch → Develop & Test → Commit Changes
→ Push Branch → Create Pull Request → Code Review → Merge
```

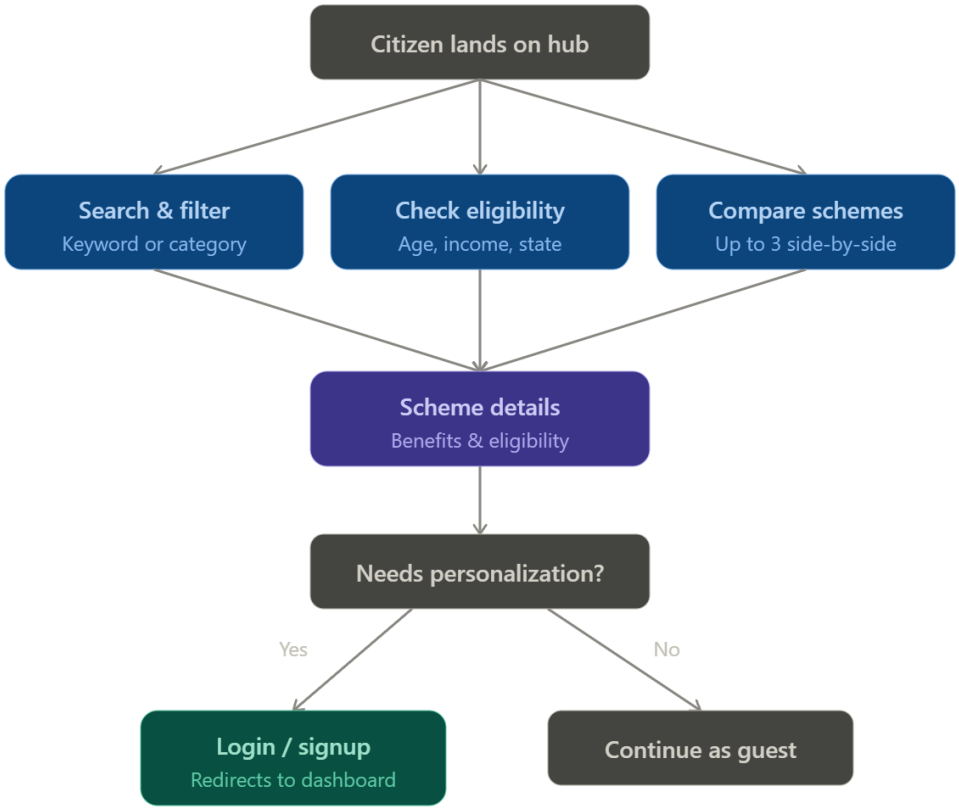
### Example Commit Messages

```
feat: add scheme comparison
fix: resolve admin routing issue
docs: update project documentation
refactor: improve authentication flow
```

# 13. Application Workflows

## 13.1 Citizen Workflow

The end-to-end journey a citizen follows within the platform:

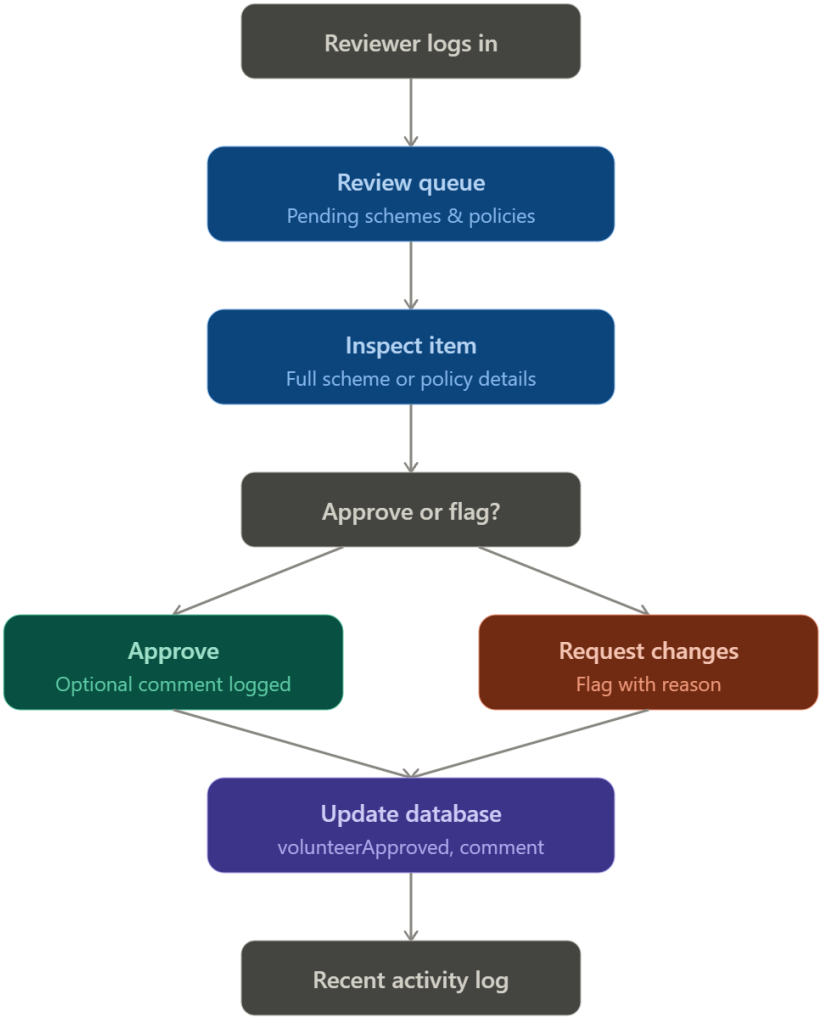



**Citizen Workflow Diagram**

```
Citizen → Register / Login → Citizen Dashboard
   → Search Schemes / Policies → View Details
        → Check Eligibility | Compare | Save
   → Receive Notifications → Submit Feedback
```

## 13.2 Volunteer Review Workflow

The review path a scheme or policy follows before publication:




***Volunteer Review Workflow Diagram***

---

```
Scheme / Policy Content → Assigned for Review → Volunteer
   → Review Information → Approve | Suggest Changes
   → Administrative Workflow
```

# 14. Application Screens

Each major screen below should be paired with an actual screenshot from the running application. Placeholders are provided in the recommended order.

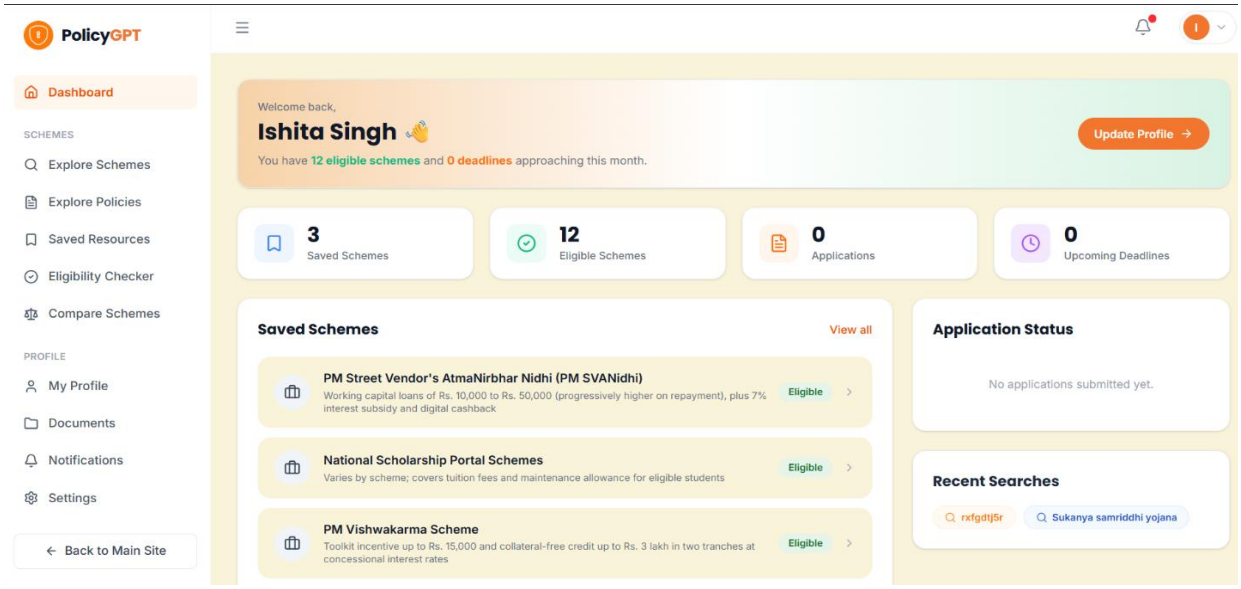
## 14.1 Landing Page

Purpose: Introduces PolicyGPT to visitors and provides entry points for login/registration.



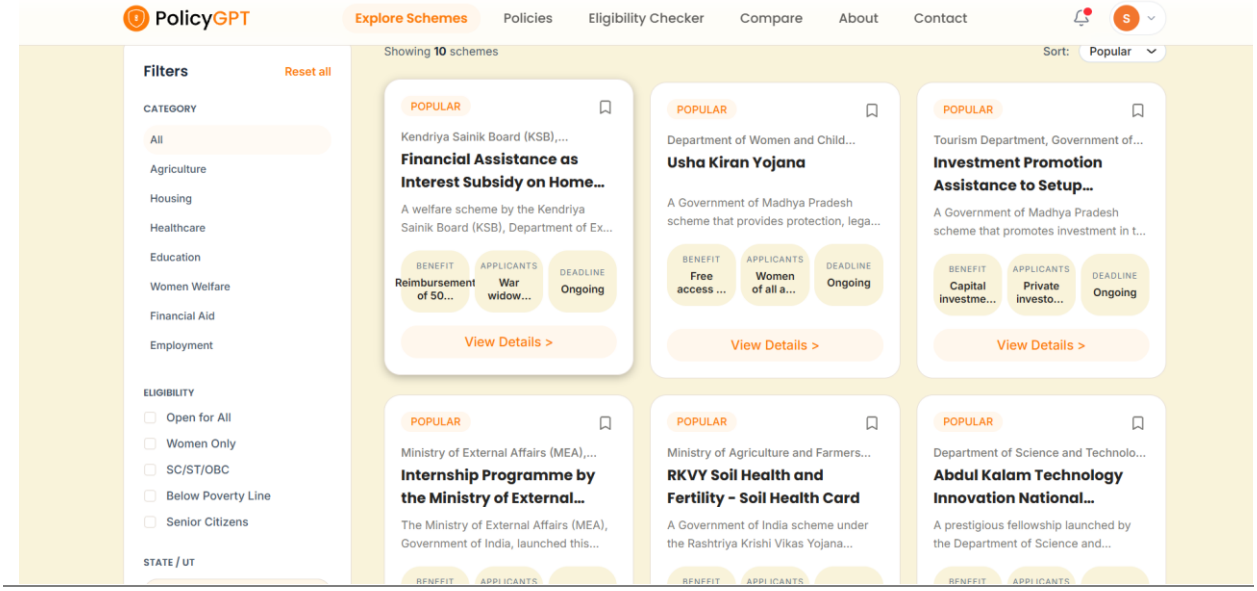

## 14.2 Citizen Dashboard

Purpose: Central workspace for citizens to access government schemes, policies, eligibility checking, saved information, and other relevant features.



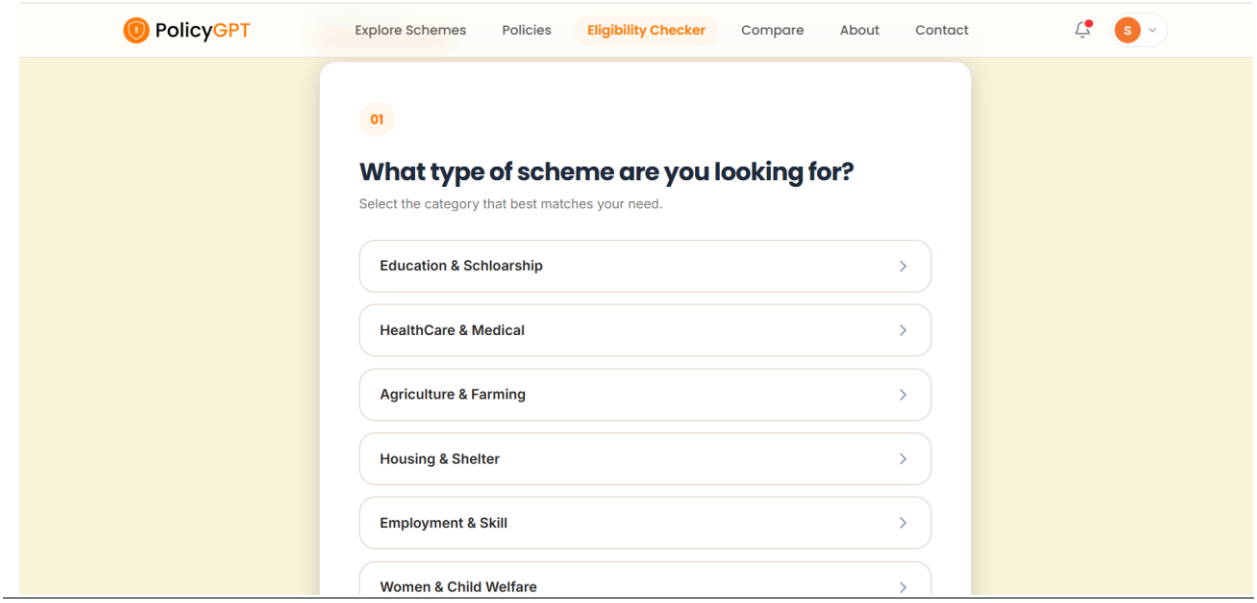
## 14.3 Scheme Details

Purpose: Displays complete information about a selected scheme, including benefits, eligibility, and department.



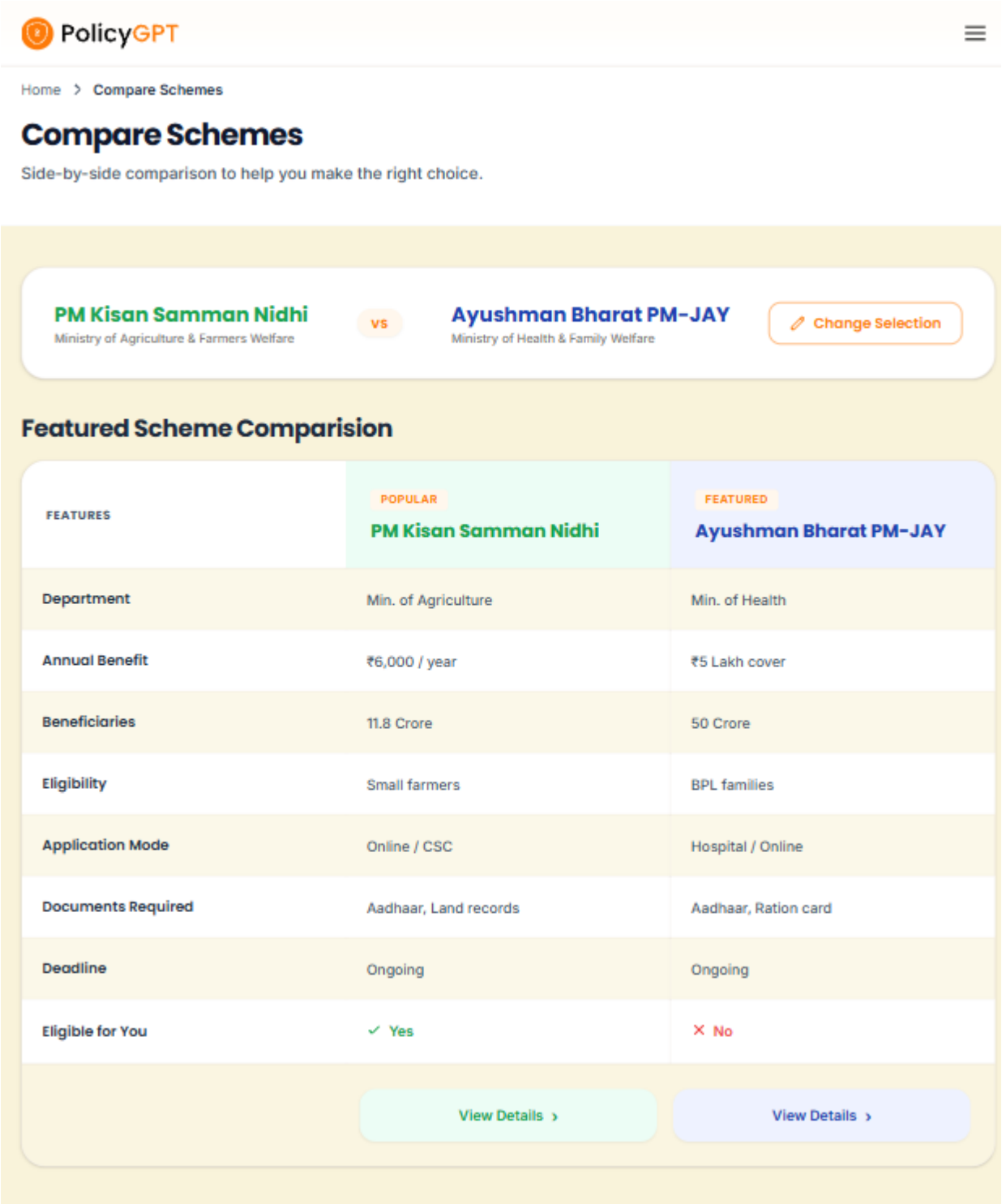

## 14.4 Eligibility Checker

Purpose: Lets citizens enter their information and check eligibility against scheme requirements.



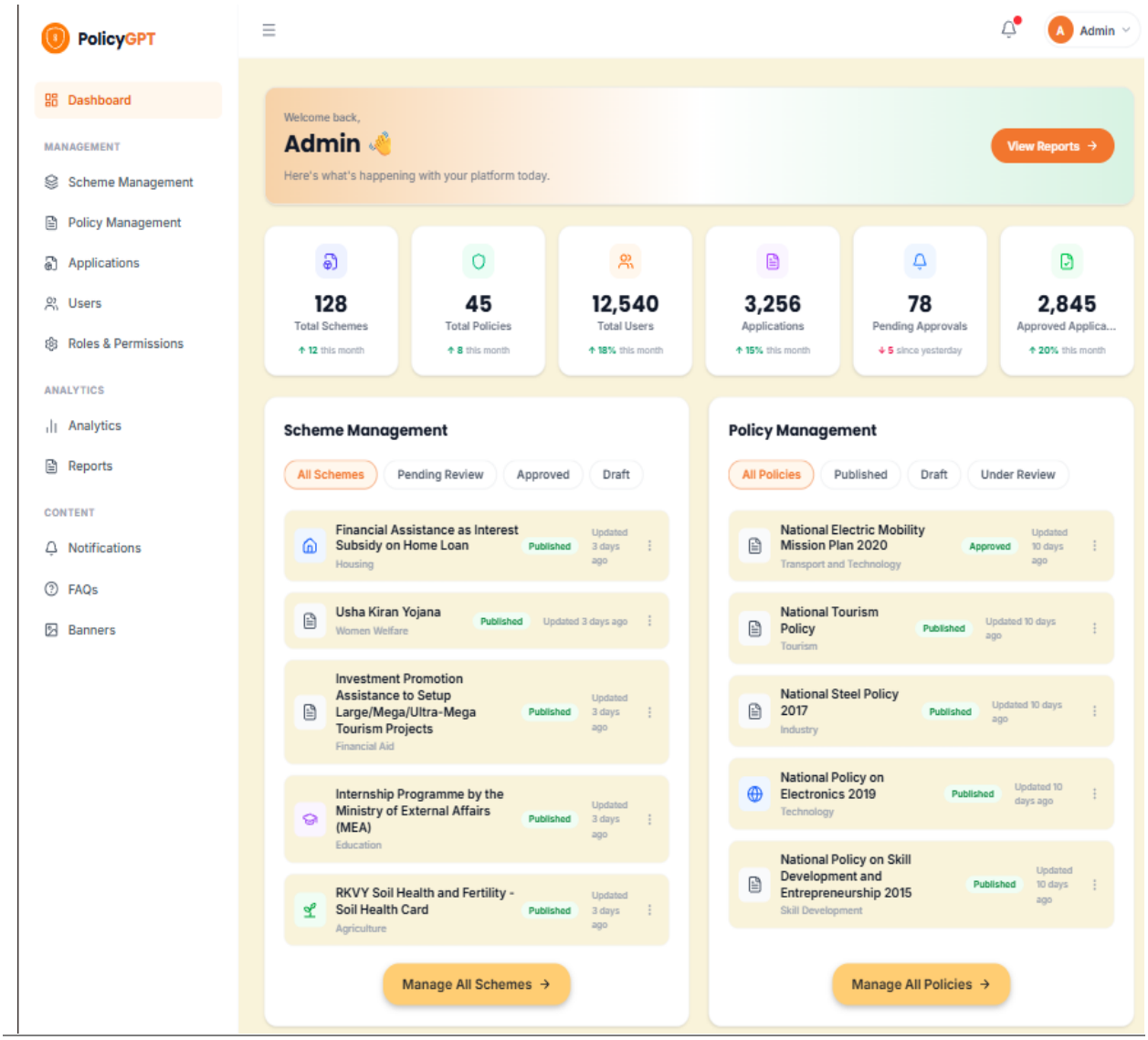
### 14.5 Scheme Comparison

Purpose: Enables citizens to compare multiple schemes side by side.



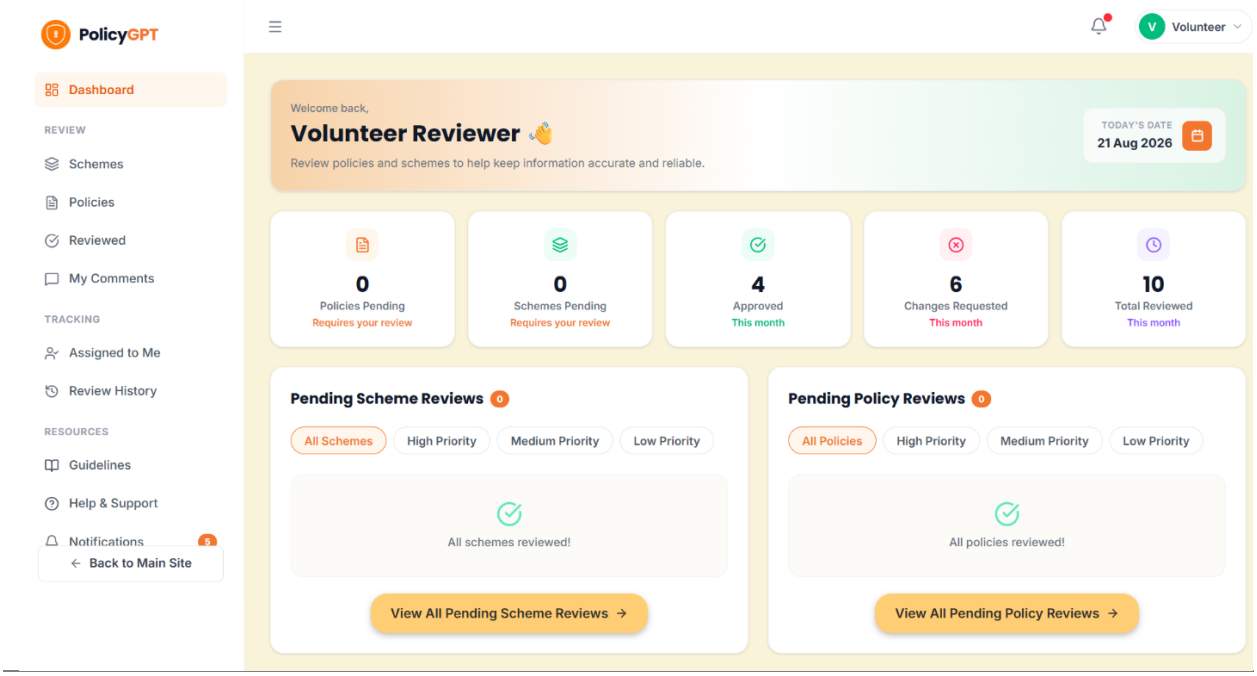
## 14.6 Admin Dashboard

Purpose: Central control panel for administrators to manage users, schemes, policies, notifications, analytics, and reports.



### 14.7 Volunteer Dashboard

Purpose: Workspace for volunteers/reviewers to review submitted schemes and policies.




## 15. API Documentation

Expected API categories, based on documented platform features:

- Authentication APIs
- User Management APIs
- Scheme APIs
- Policy APIs
- Eligibility APIs
- Comparison-related APIs
- Notification APIs
- Feedback APIs
- Analytics APIs
- Report APIs
- Volunteer Review APIs

## 16. Error Handling and Troubleshooting

### 16.1 Frontend Issues

- Ensure dependencies are installed (npm install)
- Check the Vite development server is running
- Check browser console for errors
- Verify API communication with the backend

### 16.2 Backend Issues

- Ensure dependencies are installed
- Check whether the backend server is running
- Verify environment configuration
- Check server logs
- Verify API routes and middleware

### 16.3 Database Connection Issues

- Verify the MongoDB connection
- Check environment variables
- Ensure MongoDB / MongoDB Atlas is accessible
- Check database credentials and connection configuration

### 16.4 Authentication Issues

- Verify JWT configuration
- Check authentication middleware
- Ensure valid user credentials are being used
- Check role-based access restrictions

## 17. Future Enhancements

- Enhanced AI assistance for scheme and policy information processing
- Improved search and discovery mechanisms
- Expanded analytics and reporting capabilities
- Improved, more intelligent scheme recommendations
- Additional government information sources
- Enhanced notification system with more channels

## 18. Maintenance and Support

- Monitor application performance
- Review backend and frontend logs
- Maintain database backups

- Update dependencies regularly
- Review security configurations
- Manage environment variables securely
- Test new features before merging
- Review pull requests and code changes
- Monitor authentication and authorization functionality

## 19. Project Status

### Current Status: Active Development

PolicyGPT is being developed iteratively as part of the Infosys Springboard Internship 7.0. Features are implemented, tested, integrated, and refined throughout the project lifecycle.

## 20. Core Contributors

| Contributor | Contribution |
|---|---|
| Ishita Singh | UI/UX Design, Figma Wireframes, Frontend Development, Volunteer Dashboard, Documentation |
| Souvik Chandra | UI/UX Design and Design Support |
| Sheetal Gourh | Backend Development, API Integration, Database Management, Documentation |
| Prachi Kumari | Citizen Dashboard Development |
| Ankesh Agarwal | Admin Dashboard Development |
| Mythreyee S | Schemes Module Development |
| Sai Mounika Chunchu | Policies Module Development |

## 21. Conclusion

PolicyGPT is a centralized Government Policy & Public Scheme Intelligence Platform developed to make government information easier to discover, understand, and access. The platform provides role-based functionality for citizens, administrators, and volunteers: citizens can explore schemes and policies, search and filter information, check eligibility, compare schemes, save relevant information, receive notifications, and submit feedback; administrators can manage platform content and monitor activity; and volunteers participate in reviewing schemes and policies.

The project is built on a full-stack architecture using React, Vite, Tailwind CSS, Node.js, Express.js, MongoDB, and Mongoose, with AI-assisted data enrichment powered by Google Gemini and Google Search grounding.

## 22. Next Steps for a Fully Detailed Version

To match the depth of a complete technical reference, the following sections should be filled in using the actual project source code rather than general descriptions:

- Real API endpoints — extracted from Server/routes and Server/controllers
- Complete database schemas — extracted from Server/models
- Detailed authentication flow
- Detailed notification flow
- Eligibility-checking logic
- Actual AI / RAG workflow used with Gemini
- Real environment variable names
- Screenshots for every screen listed in Section 14